CS 577 Deep Learning
Homework 3: Attention and Transformer Forward

Your Name (Hawk username: `yourname`)

Fall 2026

Part 0: Setup and Axes

All experiments were executed on a CPU with a fixed random seed of `577`. The notebook environment used the following versions:

- Python: `3.11.18`
- NumPy: `2.4.6`
- PyTorch: `2.15.0`
- Matplotlib: `3.11.2`
- Device: `CPU`

All 11 public tests passed successfully. These tests covered stable masked softmax, causal masking, scaled dot-product attention, head splitting and merging, multi-head self-attention and cross-attention, positional encodings, and Transformer block behavior.

The final test summary was:

```
Ran 11 tests in 0.047s

OK

Public-test return code: 0
```

No failures or errors were reported.

0.2 Axis and mask contract

The attention scores S, attention weights A, and output Y have the following shapes:

$$S : (B, H, T_q, T_k), \qquad A : (B, H, T_q, T_k), \qquad Y : (B, H, T_q, d_v).$$

Here, B is the batch size, H is the number of heads, T_q is the number of query tokens, T_k is the number of key tokens, d_k is the query/key feature size, and d_v is the value feature size.

We apply softmax along the last axis (T_k) because each query compares itself with all keys. Softmax gives a probability to each allowed key, and these probabilities add up to 1. If all keys are blocked, the probabilities are all zero.

The key mask first has shape (B, T_k). We reshape it to $(B, 1, 1, T_k)$ because each batch may have a different mask, but the same mask is used for all heads and queries within that batch.

NumPy broadcasting works by matching dimensions from right to left. A dimension of size 1 can expand to match the corresponding dimension in another array.

This allows the mask with shape $(B, 1, 1, T_k)$ to work with attention scores of shape (B, H, T_q, T_k).

Part 1: Hand Calculation

The given query, key, and value matrices are:

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}.$$

All three matrices have shape $(3, 2)$, and $d_k = d_v = 2$.

First, we calculate the dot products between queries and keys:

$$QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

The shape of $QK^\top$ is $(3, 3)$.

Next, we divide each score by $\sqrt{d_k} = \sqrt{2}$:

$$S = \frac{QK^\top}{\sqrt{2}} = \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

The scaled score matrix S has shape $(3, 3)$.

We apply softmax along each row to calculate the attention weights:

$$A = \text{softmax}(S) = \begin{bmatrix} 0.4011 & 0.1978 & 0.4011 \\ 0.1978 & 0.4011 & 0.4011 \\ 0.2483 & 0.2483 & 0.5035 \end{bmatrix}.$$

The attention weight matrix A has shape $(3, 3)$. Each row sums to one.

Softmax calculation for cat. The scaled scores for the cat row are:

$$s_{\text{cat}} = \left[0, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right].$$

The softmax formula is:

$$a_j = \frac{e^{s_j}}{\sum_k e^{s_k}}.$$

The common denominator is:

$$e^0 + e^{1/\sqrt{2}} + e^{1/\sqrt{2}}.$$

Using this denominator, the attention weights are:

$$a_{\text{The}} = \frac{e^0}{e^0 + 2e^{1/\sqrt{2}}} \approx 0.1978,$$

$$a_{\text{cat}} = a_{\text{sleeps}} = \frac{e^{1/\sqrt{2}}}{e^0 + 2e^{1/\sqrt{2}}} \approx 0.4011.$$

The attention weights for cat are:

$$a_{\text{cat,row}} = [0.1978,\ 0.4011,\ 0.4011].$$

Attention output. The attention output is calculated by multiplying the attention weights by the value matrix:

$$Y = AV = \begin{bmatrix} 1.6044 & 0.7967 \\ 1.4011 & 1.2033 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

The output matrix Y has shape $(3, 2)$.

Weighted sum for cat. The value vectors are:

$$V_{\text{The}} = [1, 0], \qquad V_{\text{cat}} = [0, 2], \qquad V_{\text{sleeps}} = [3, 1].$$

We multiply each value vector by its attention weight and add the results:

$$Y_{\text{cat}} = 0.1978[1, 0] + 0.4011[0, 2] + 0.4011[3, 1].$$

Calculating each output component separately gives:

$$Y_{\text{cat}} = [0.1978(1) + 0.4011(0) + 0.4011(3),$$

$$0.1978(0) + 0.4011(2) + 0.4011(1)]\,.$$

The final output for cat is:

$$\boxed{Y_{\text{cat}} = [1.4011,\ 1.2033]}$$

1.2 Causal attention

In causal attention, each query can attend only to the current token and previous tokens. Future tokens are blocked.

In our example:

- The can attend only to The.
- cat can attend to The and cat.
- sleeps can attend to all three tokens.

We use the causal mask M, where 1 means allowed and 0 means blocked:

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

The mask has shape $(3, 3)$.

After applying the mask and softmax, the attention weights are:

$$A_{\text{causal}} = \begin{bmatrix} 1.0000 & 0 & 0 \\ 0.3302 & 0.6698 & 0 \\ 0.2483 & 0.2483 & 0.5035 \end{bmatrix}.$$

The weight matrix has shape $(3, 3)$. Blocked keys have attention weights of exactly zero.

The new attention output is:

$$Y_{\text{causal}} = A_{\text{causal}} V = \begin{bmatrix} 1.0000 & 0 \\ 0.3302 & 1.3395 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

The output matrix has shape $(3, 2)$.

Weighted sum for cat. The cat query can attend only to The and cat. The sleeps token is blocked.

The attention weights for cat are:

$$a_{\text{cat}} = [0.3302,\ 0.6698,\ 0].$$

Using the value vectors, we calculate:

$$Y_{\text{cat}} = 0.3302[1, 0] + 0.6698[0, 2] + 0[3, 1].$$

Calculating the two output components separately:

$$\begin{aligned} Y_{\text{cat}} &= \big[0.3302(1) + 0.6698(0) + 0(3), \\ &\qquad 0.3302(0) + 0.6698(2) + 0(1)\big] \\ &\approx [0.3302,\ 1.3396]. \end{aligned}$$

Why the last row does not change: The last token, sleeps, can attend to all three tokens because there are no future tokens after it. Therefore, its attention weights and output remain the same as in unmasked attention.

Why each output has two values: Each value vector has two features $d_v = 2$. Attention calculates a weighted sum of these vectors. The causal mask changes which tokens are used, but it does not change the number of output features. That's why each output vector still has two values.

1.3 Scaling and stability

The dot product between a query and a key is:

$$s = q^\top k = \sum_{r=0}^{d_k-1} q_r k_r.$$

We assume that all coordinates are mutually independent, have zero mean, and unit variance:

$$E[q_r] = E[k_r] = 0, \qquad \mathrm{Var}(q_r) = \mathrm{Var}(k_r) = 1.$$

Using the variance formula and the zero mean:

$$\begin{aligned} \mathrm{Var}(q_r) = E[q_r^2] - (E[q_r])^2 = 1, \\ E[q_r^2] = 1. \end{aligned}$$

Similarly, $E[k_r^2] = 1$.

For each product, independence gives:

$$E[q_r k_r] = E[q_r]E[k_r] = 0.$$

$$E[q_r^2 k_r^2] = E[q_r^2]E[k_r^2] = 1.$$

Therefore,

$$\begin{aligned} \mathrm{Var}(q_r k_r) &= E[(q_r k_r)^2] - (E[q_r k_r])^2 \\ &= 1 - 0^2 \\ &= 1. \end{aligned}$$

Since all coordinates are mutually independent, each product uses a different independent pair. Therefore, the variances of the products add:

$$\begin{aligned}\mathrm{Var}(s) &= \mathrm{Var}\left(\sum_{r=0}^{d_k-1} q_r k_r\right) \\ &= \sum_{r=0}^{d_k-1} \mathrm{Var}(q_r k_r) \\ &= \sum_{r=0}^{d_k-1} 1 \\ &= \boxed{d_k}.\end{aligned}$$

After scaling by $\sqrt{d_k}$:

$$s_{\text{scaled}} = \frac{s}{\sqrt{d_k}}.$$

Using $\mathrm{Var}(cX) = c^2 \mathrm{Var}(X)$:

$$\begin{aligned}\mathrm{Var}(s_{\text{scaled}}) &= \mathrm{Var}\left(\frac{s}{\sqrt{d_k}}\right) \\ &= \frac{1}{d_k}\mathrm{Var}(s) \\ &= \frac{d_k}{d_k} \\ &= \boxed{1}.\end{aligned}$$

Why scaling is important: Without scaling, the standard deviation of the attention scores is $\sqrt{d_k}$. As d_k grows, the scores can become more spread out.

For two softmax weights, their ratio is:

$$\frac{a_i}{a_j} = e^{s_i - s_j}.$$

This means the ratio grows exponentially when the difference between scores increases. As a result, the largest score can receive almost all the attention weight, while the others receive values close to zero.

Dividing by $\sqrt{d_k}$ keeps the score variance controlled and reduces this problem.

Numerical stability: For numerical stability, we subtract the maximum score from each row before applying softmax:

$$m = \max_j s_j.$$

The softmax formula becomes:

$$\begin{aligned}\text{softmax}(s_i) &= \frac{e^{s_i}}{\sum_j e^{s_j}} \\ &= \frac{e^{s_i - m} e^{m}}{\sum_j e^{s_j - m} e^{m}} \\ &= \frac{e^{s_i - m}}{\sum_j e^{s_j - m}}.\end{aligned}$$

The common factor e^m cancels, so subtracting the maximum does not change the softmax result.

After subtraction, the largest shifted score is zero, and its exponential is:

$$e^0 = 1.$$

All other shifted scores are less than or equal to zero, so their exponentials are at most one.

For a row with at least one allowed key, the denominator contains at least one term equal to one. Therefore, it is at least one, which avoids division by zero and denominator underflow.

For example, consider the scores:

$$[1000,\ 999,\ -1000].$$

Computing e^{1000} directly can cause overflow. After subtracting the maximum, we obtain:

$$[0,\ -1,\ -2000].$$

The exponentials are now:

$$[1,\ e^{-1},\ e^{-2000}].$$

This makes the calculation numerically stable without changing the softmax probabilities.

A fully masked row is handled separately by returning zeros, as required by the implementation.

Part 2: Stable Masked Attention

2.1 Numerical stability and fully blocked rows

I implemented masked softmax using NumPy. In my code, True means allowed and False means blocked.

The mask is applied before normalization. Blocked positions receive attention weights of exactly zero.

To avoid overflow, I subtract the largest allowed score in each row before calculating the exponential.

$$A_{ij} = \begin{cases} \dfrac{e^{S_{ij} - m_i}}{\sum_{r \text{ allowed}} e^{S_{ir} - m_i}}, & \text{if } j \text{ is allowed,} \\ 0, & \text{if } j \text{ is blocked.} \end{cases}$$

Here, m_i is the largest allowed score in row i.

For a fully blocked row, the maximum would be $-\infty$. Subtracting it from itself would produce NaN. To prevent this, my code replaces a non-finite row maximum with zero before calculating the exponential.

It also uses a safe denominator and returns zeros for the fully blocked row.

I tested extreme scores, including $[1000, 999, -1000]$, and a fully blocked row.

The results were:

$$A = \begin{bmatrix} 0.7311 & 0.2689 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

The row sums were:

$$[1.0,\ 0.0].$$

All results were finite, with no warnings or NaN values.

My code raises ValueError if a mask is not Boolean or cannot broadcast to the score shape.

2.2 Hand-case agreement

I implemented scaled dot-product attention using NumPy and the following formulas:

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad A = \text{masked_softmax}(S), \qquad Y = AV.$$

The input shapes are:

$$Q : (B, H, T_q, d_k),$$
$$K : (B, H, T_k, d_k),$$
$$V : (B, H, T_k, d_v).$$

The function returns (Y, cache), where:

$$Y : (B, H, T_q, d_v).$$

The cache contains the scaled scores before masking and the attention weights. Both have shape:

$$(B, H, T_q, T_k).$$

The implementation supports rectangular attention ($T_q \neq T_k$) and different feature sizes ($d_k \neq d_v$).

It raises `ValueError` when input dimensions are not compatible.

Hand-case results. I tested my implementation using the provided `hand_case()` with three tokens: The, cat, and sleeps.

The inputs have shape $(1, 1, 3, 2)$, which means one batch, one attention head, three tokens, and two features.

For unmasked attention, I obtained:

$$Y_{\text{full}} = \begin{bmatrix} 1.6044 & 0.7967 \\ 1.4011 & 1.2033 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

For causal attention, I obtained:

$$Y_{\text{causal}} = \begin{bmatrix} 1.0000 & 0 \\ 0.3302 & 1.3395 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

Both outputs have shape $(1, 1, 3, 2)$. The matrices above omit the batch and head axes for easier reading.

Numerical agreement. I compared the NumPy results with the values written to four decimal places in Part 1.

The maximum absolute differences, based on the displayed NumPy values, were approximately:

$$\text{Unmasked difference} \approx 4.8 \times 10^{-5},$$
$$\text{Causal difference} \approx 3.8 \times 10^{-5}.$$

Both differences are below 5×10^{-5}, which is the expected rounding difference for four decimal places.

The calculations in NumPy use full precision.

2.3 Padding and causal masks

I tested two sequences with a maximum length of four tokens. The first sequence has two real tokens, and the second has four real tokens.

The padding mask has shape $(B, 1, 1, T)$. The causal mask has shape $(1, 1, T, T)$.

I combined them using Boolean AND:

$$M_{\text{combined}} = M_{\text{padding}} \wedge M_{\text{causal}}.$$

The combined mask has shape $(B, 1, T, T)$.

For the first sequence, with two real tokens:

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}.$$

For the second sequence, with four real tokens:

$$M_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

Here, 1 means allowed, and 0 means blocked.

In the first sequence, positions 2 and 3 are padding, so they are blocked as keys. The causal mask also blocks future keys.

I checked all blocked positions in the attention weights. The largest blocked weight was:

$$\boxed{0.0}$$

This shows that padded and future keys received zero attention weight.

Next, I multiplied the padded value vectors by a large number and checked whether the outputs for real query tokens changed.

The maximum changes were:

$$\boxed{[0.0,\ 0.0]}$$

The outputs did not change. This shows that the padded values did not affect the results for real tokens.

I tested fully blocked rows separately in Section 2.1. They returned zeros without warnings or NaN values.

Mask rules in PyTorch.

My NumPy code uses True for allowed keys.

In `torch.nn.MultiheadAttention`, True means blocked. Therefore, I invert my Boolean mask when using this PyTorch function.

The PyTorch key-padding mask also uses True for padding.

In `torch.nn.functional.scaled_dot_product_attention`, True means allowed, just like in my NumPy code.

For my PyTorch reference, I used `torch.nn.MultiheadAttention` and inverted the allowed mask.

I compared only rows with at least one allowed key because fully blocked rows need special handling.

Part 3: Multi-Head Attention

3.1 Shape trace, attention maps, and outputs

I used the provided token embeddings for The, cat, and sleeps:

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}.$$

The input has one batch, three tokens, and four features:

$$B = 1, \quad T = 3, \quad D = 4.$$

I used two attention heads:

$$H = 2, \qquad d_h = \frac{D}{H} = 2.$$

I initialized the projection matrices using `initialize_mha(4, seed=577)`.

First, I projected the complete input using the query, key, and value matrices:

$$Q_{\text{full}} = XW_Q, \qquad K_{\text{full}} = XW_K, \qquad V_{\text{full}} = XW_V.$$

Next, I split the projected features into two heads and calculated attention separately for each head.

The shapes at each step were:

Stage	Meaning	Shape
X	Input features	$(1, 3, 4)$
XW_Q, XW_K, XW_V	Projected features	$(1, 3, 4)$
Q, K, V	Split heads	$(1, 2, 3, 2)$
S, A	Scores and weights	$(1, 2, 3, 3)$
Head outputs	Weighted values	$(1, 2, 3, 2)$
Merged outputs	Joined heads	$(1, 3, 4)$
Y_{MHA}	Final output	$(1, 3, 4)$

Each head calculates its own attention weights. The scaling factor is:

$$\sqrt{d_h} = \sqrt{2}.$$

The attention weights for Head 0 were:

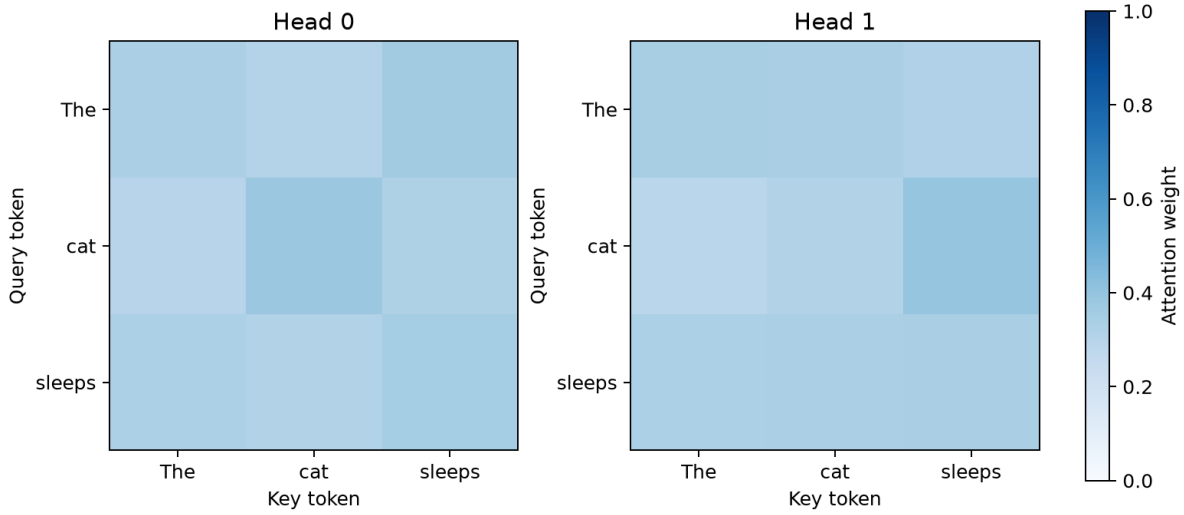
$$A_0 = \begin{bmatrix} 0.3336 & 0.3057 & 0.3607 \\ 0.2945 & 0.3826 & 0.3229 \\ 0.3310 & 0.3147 & 0.3544 \end{bmatrix}.$$

The attention weights for Head 1 were:

$$A_1 = \begin{bmatrix} 0.3433 & 0.3370 & 0.3198 \\ 0.2884 & 0.3139 & 0.3978 \\ 0.3293 & 0.3346 & 0.3361 \end{bmatrix}.$$

Both attention weight matrices have shape $(3, 3)$.

I plotted the weights for both heads. The rows show query tokens, and the columns show key tokens. Both plots use the same color scale from zero to one.



After calculating attention, I merged the two head outputs and applied the output projection:

$$Y_{\text{MHA}} = \text{Concat}(Y_0, Y_1)W_O.$$

The final output was:

$$Y_{\text{MHA}} = \begin{bmatrix} -0.8025 & 0.2970 & -0.2950 & 0.2870 \\ -0.7275 & 0.4051 & -0.2322 & 0.2978 \\ -0.7932 & 0.3091 & -0.2887 & 0.2858 \end{bmatrix}.$$

The final output has shape $(1, 3, 4)$. The matrix above shows the single batch.

Why projection comes before splitting.

The projection matrices use all four input features to calculate new query, key, and value features.

If I split the original input before projection, each head would only receive part of the original features.

This would not be the same as using the learned full-width projections, because the projections can mix information from all input features. **Attention map interpretation:** The attention weights in both heads are mostly between 0.29 and 0.40. The heads give similar attention to all three words, without strongly focusing on one word. This makes sense because the projection matrices have random values and have not been trained to learn language patterns.

3.2 Independent PyTorch comparison

I compared my NumPy implementation with `torch.nn.MultiheadAttention`.

I used the following settings:

```
embed_dim = 4
num_heads = 2
bias = False
dropout = 0.0
```

```
batch_first = True
dtype = torch.float64
device = CPU
```

PyTorch stores linear weights in transposed form compared with my NumPy implementation.

I copied the query, key, and value matrices into PyTorch by stacking their transposes:

$$W_{\text{in}} = \begin{bmatrix} W_Q^\top \\ W_K^\top \\ W_V^\top \end{bmatrix}.$$

For the output projection, I used:

$$W_{\text{out}} = W_O^\top.$$

I used `average_attn_weights=False` to compare the weights for each head separately.

My NumPy code uses True for allowed positions. In PyTorch's `nn.MultiheadAttention`, True means blocked, so I inverted the mask.

I tested three cases: unmasked self-attention, causal self-attention, and rectangular cross-attention.

The cross-attention test used two query tokens and three context tokens.

The maximum absolute errors were:

Case	Output error	Weight error
Unmasked self	1.11×10^{-16}	5.55×10^{-17}
Causal self	1.11×10^{-16}	5.55×10^{-17}
Rectangular cross	1.11×10^{-16}	5.55×10^{-17}

All errors were smaller than 10^{-9}. This shows that my NumPy implementation agrees with the independent PyTorch reference within floating-point precision.

3.3 Head count and head-output ablation

I compared one attention head with two heads. I kept the input and all four projection matrices unchanged.

The results were:

Property	$H = 1$	$H = 2$
Features per head	4	2
Score shape	$(1, 1, 3, 3)$	$(1, 2, 3, 3)$
MHA parameters	64	64

The maximum output difference between one head and two heads was:

$$\boxed{0.0397}$$

Why the parameter count is the same: The total feature width stays $D = 4$. There are four projection matrices:

$$W_Q, W_K, W_V, W_O \in \mathbb{R}^{4\times 4}.$$

Each matrix has:

$$4 \times 4 = 16$$

parameters.

The total parameter count is:

$$4 \times 16 = \boxed{64}.$$

Changing the number of heads changes how the features are divided, but it does not change the size of these projection matrices.

Why the outputs can differ: With one head, attention uses all four projected features together.

With two heads, each head uses two features and calculates its own attention weights.

Because the score calculations and attention weights are different, the final outputs can also be different.

Removing one head: I tested the effect of removing each head. I set one head's output to zero before merging the heads and applying W_O.

First, I checked that merging without removing a head gave the same output:

$$\text{Maximum difference} = 0.0.$$

Then I removed each head separately.

The maximum output changes were:

Experiment	Maximum output change
Remove Head 0	0.4549
Remove Head 1	0.4669

Both heads affect the final output. Removing either head changes the result.

Head 1 caused a slightly larger maximum change in this example. However, this does not prove that Head 1 is more important in general.

The projection matrices in this experiment are fixed sample parameters and were not trained on a language task.

Therefore, these attention maps do not prove that either head learned a specific meaning or language role. **Why the outputs can differ:** With one head, all four features are used together to calculate attention weights. The scores are divided by $\sqrt{4} = 2$, and softmax is applied once.

With two heads, the features are split into two groups. Each head uses two features, divides the scores by $\sqrt{2}$, and calculates its own softmax.

Softmax is not a linear function, so two separate softmax calculations do not give the same result as one combined calculation.

That is why changing the number of heads can change the final output, even when the input and projection matrices stay the same.

Part 4: Transformer Block

4.1 Shape trace and reference checks

TODO

4.2 Positions and reordering

TODO

4.3 Future perturbation and causal visibility

TODO

Part 5: Architecture Comparison

5.1 Encoder–decoder and decoder-only generation

TODO

Assistance and Generative-AI Disclosure

TODO